In [2]:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load and prepare data
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# --- ENTER YOUR CODE HERE ---
# 1. Create and train a single Decision Tree
single_tree = DecisionTreeClassifier(random_state=42)
single_tree.fit(X_train, y_train)

# 2. Make predictions and evaluate its accuracy
y_pred_tree = single_tree.predict(X_test)
accuracy_tree = accuracy_score(y_test, y_pred_tree)
print(f"Accuracy of a single Decision Tree: {accuracy_tree:.2%}")

# --- ENTER YOUR CODE HERE ---
# 1. Create and train a Random Forest
#    `n_estimators` is the number of trees in the forest.
random_forest = RandomForestClassifier(n_estimators=100, random_state=42)
random_forest.fit(X_train, y_train)

# 2. Make predictions and evaluate its accuracy
y_pred_rf = random_forest.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print(f"Accuracy of the Random Forest: {accuracy_rf:.2%}")

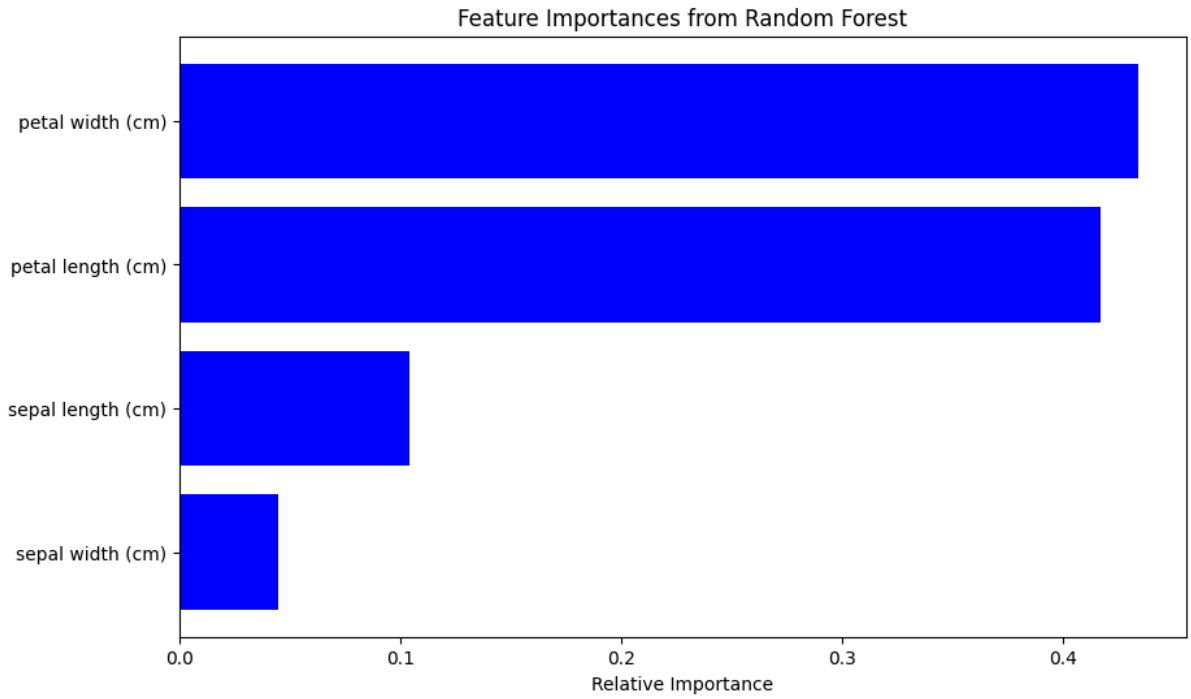
import matplotlib.pyplot as plt
import numpy as np

# --- ENTER YOUR CODE HERE ---
# 1. Get the feature importances from the trained Random Forest model
importances = random_forest.feature_importances_

# 2. Create a bar plot to visualize them
feature_names = iris.feature_names
indices = np.argsort(importances)
plt.figure(figsize=(10, 6))
plt.title("Feature Importances from Random Forest")
plt.barh(range(len(indices)), importances[indices], color='b',
align='center')
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
```

```
plt.xlabel('Relative Importance')
plt.show()
```

```
Accuracy of a single Decision Tree: 100.00%
Accuracy of the Random Forest: 100.00%
```



📝 Knowledge Check

1. In your own words, what is the main idea behind ensemble methods? Why is a "committee" of models often better than a single expert model?

By creating an ensemble model, multiple models are used. An example is the "committee" method where several models are trained, each on different training data, to predict results. Using several models, the variance is reduced and the ensemble results are better than those of a single model. This is due to each individual model's bias. Results become more consistent and reliable due to the errors of individual models cancelling each other out. This approach creates an even more powerful and accurate model.

2. Which model performed better in this lab, the single Decision Tree or the Random Forest? Is this what you expected? Why?

In this lab, a single Decision Tree and a Random Forest were applied to the test set of the Iris dataset. Both algorithms achieved a 100% accuracy score. There was no improvement in the raw accuracy score of both models because of the small dataset. In this situation, the dataset was very clean and had distinct classes. However, it is common knowledge in machine learning that Random Forests work better than single Decision Trees on more noisy and complex real world data because of their ability to generalize.

3. Looking at the feature importance plot, which two features were most important for classifying the iris flowers? Does this match what you might expect from your own knowledge of flowers?

The feature importance graph demonstrates that the most useful attributes for classifying an iris flower are petal length and petal width, which is the most distinguishing characteristic over all sepal attributes combined. Petal length and petal width vary widely among different species of irises. Because of this, petals tend to provide clearly defined edges, and therefore contain the most information that is useful to differentiate among classes. Conversely, across most iris species, the sepals look fairly similar, and provide the least amount of information to distinguish among classes.